

PROJECT REPORT

NIKHIL AGARWAL

TASK MANAGEMENT APP

REGISTRATION NUMBER

25BA110930

Introduction

Everyone makes to-do lists, but most of us either forget where we wrote them or end up rewriting the same list multiple times. To solve this everyday inconvenience, I decided to build a simple yet practical **Task Manager desktop application** using Python's Tkinter library.

The intention was not just to write working code, but to translate classroom concepts into a small, usable tool that genuinely improves daily productivity. This project allowed me to experiment with UI components, event-driven programming, and basic data handling inside a standalone desktop application.

Problem Statement

People often struggle to keep track of small tasks — assignments, reminders, chores, shopping lists, etc. Mobile apps exist, but they are usually bloated, require login, or bombard users with ads.

The problem I aimed to address was:

How can I design a simple, offline, distraction-free desktop tool where users can quickly add, update, search, and delete tasks?

The software should be straightforward, lightweight, and suitable even for users with minimal technical knowledge.

Functional Requirements

Based on the required three functional modules stated in the guidelines

BuildYourOwnProject and
analyzing my implementation b
, the application provides:

1. Task Input & Creation Module

- Allows users to enter a new task into the system.
- Validates empty input and alerts the user.
- Immediately updates the displayed task list.

2. Task Management Module

Includes:

- **Update functionality:** Users can select an existing task, edit it in the input box, and update it.
- **Delete functionality:** Users can remove unwanted tasks from the list.

3. Search & Filtering Module

- As the user types in the search bar, the list dynamically filters tasks.
- Case-insensitive matching.
- Displays only tasks that contain the search keyword.

Input/Output Structure

- **Input:** User-typed text (new tasks, updated tasks, search keywords).
- **Output:** Updated visual task list reflecting current items.

Non-Functional Requirements

In accordance with project guidelines, at least four non-functional requirements are needed
BuildYourOwnProject

. My project satisfies:

1. Usability

- Clean, dark-themed UI.
- Clear labels, large buttons, and simple workflow.
- No prior technical knowledge required.

2. Performance

- Since data is stored in memory, operations like search, add, and delete are instantaneous.

3. Reliability

- Error boxes prevent crashes caused by empty fields or no selection.
- UI consistently updates state after every action.

4. Maintainability

- Code is modular with separated methods (add_task(), update_task(), etc.).
- Color themes and UI structure grouped for easy future modification.

Error Handling

- Warning messages for invalid inputs.
- Error dialogs for cases like updating without selecting a task.

System Architecture

The system follows a simple event-driven architecture:

User Action → Tkinter Event → Backend Logic → UI Update

Layers

1. **Interface Layer**
Entry fields, labels, buttons, listbox.
2. **Application Logic**
Add, update, delete, and search logic.
3. **Data Layer**
In-memory Python list (self.all_tasks).

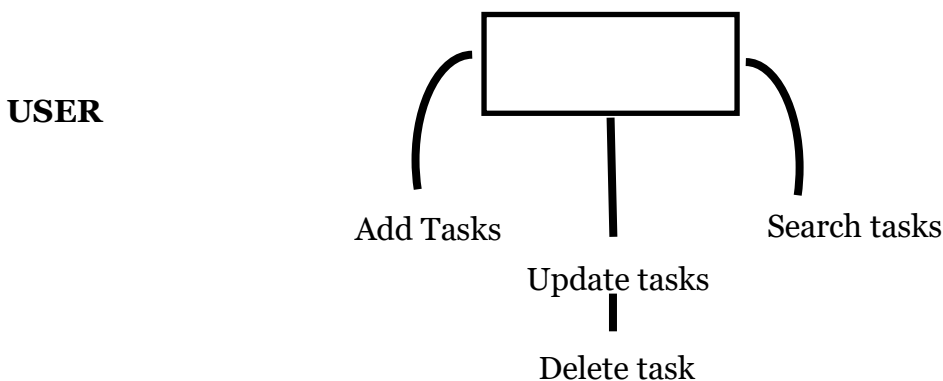
Although small, this layered structure ensures clarity and easy future enhancements

Design Diagrams

1. Use Case Diagrams

Performer: User

Use Cases: Add Task, Update Task, Delete Task, View Tasks, Search Tasks



2. Workflow Diagram

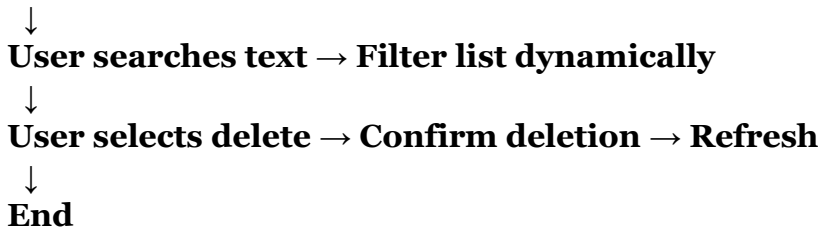
Start



User enters task → Validate → Add to list → Refresh Display



User selects task → Edits → Update List → Refresh Display



3. Sequence Diagram (Simplified)

User → UI: Type New Task
UI → Logic: add_task()
Logic → List: append()
List → UI: refresh_listbox()

4 . Class Diagram



Design Decisions & Rationale

- **Tkinter** was chosen because it's lightweight and ideal for desktop utilities.
- **Listbox + Scrollbar** allows intuitive task navigation.
- **Dark theme** is easier on the eyes and gives the application a modern appearance.
- **In-memory list** avoids complexity but keeps the project simple for learning and demonstration.

Implementation Details

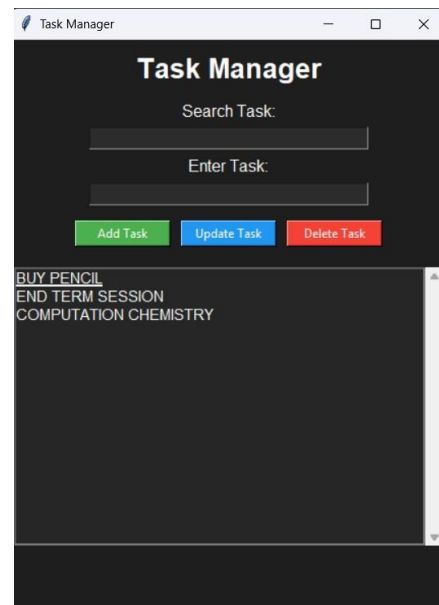
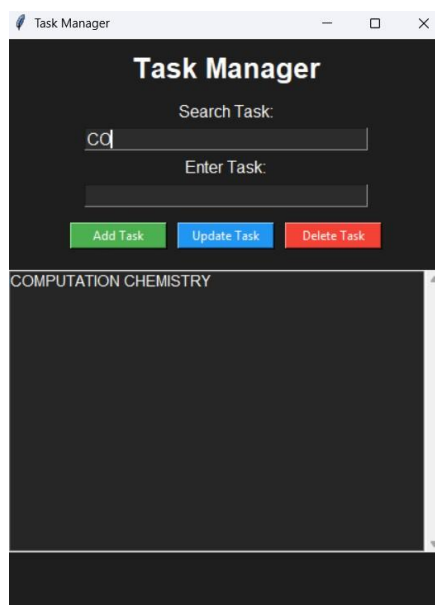
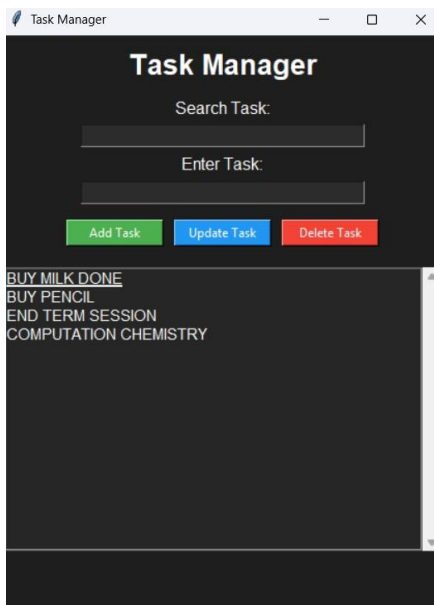
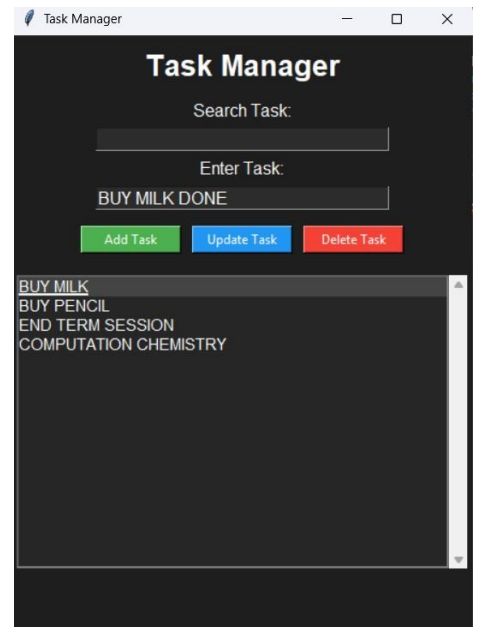
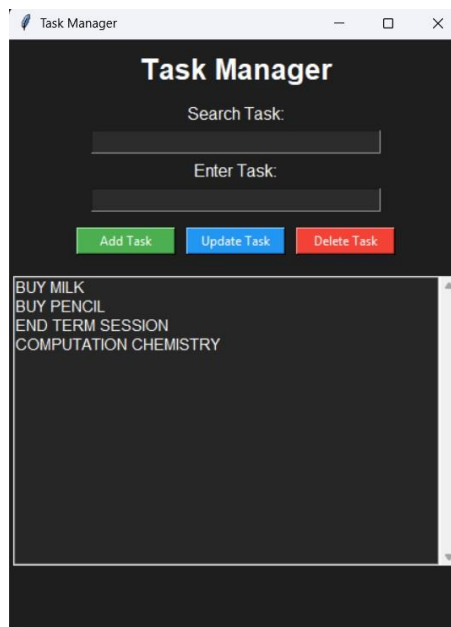
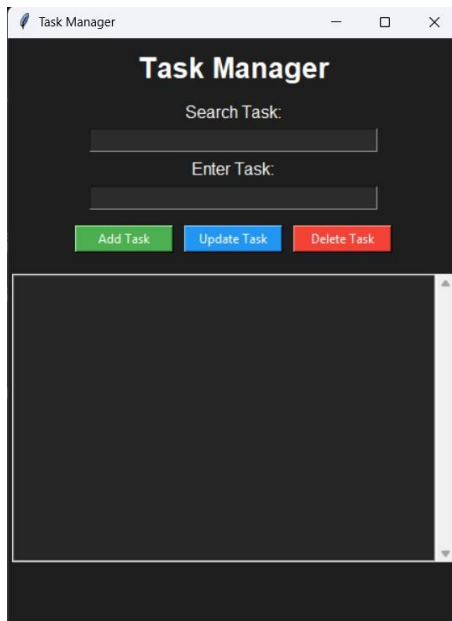
The application is implemented in Python using Tkinter Key highlights:

- The main interface is created inside a single class, ensuring modularity.
- Event binding (<KeyRelease>) enables real-time search.
- Data consistency is preserved by refreshing the listbox after every change.

- Message boxes handle validation and feedback.

Screenshots / Results

- Home screen
- Adding a task
- Updating a task
- Deleting a task
- Search filtering



Testing Approach

Basic manual testing was performed:

Test Case	Input	Expected Result	Output
-----------	-------	-----------------	--------

Add task **"Buy milk"** Task appears in list **Success** **Add**
empty task "" **Warning popup** **Success**
Update task **select + edit** Updated in list **Success**
Delete task select **Task removed** **Success**
Search **keyword** **List filters** **Success**

All core features behaved as expected.

Challenges Faced

- Initially struggled with keeping UI updates in sync with the underlying list.
 - Mapping selected filtered tasks back to the original list required careful indexing.
 - Designing a clean UI layout using Tkinter's geometry management took trial-and-error.
-

Learnings & Key Takeaways

- Improved understanding of event-driven programming.
 - Learned practical Tkinter component integration.
 - Understood the importance of error handling in user-facing applications.
 - Realized how even a simple project benefits from modular design.
-

Future Enhancements

- Add data persistence (saving tasks to a file or database).
 - Introduce categories, deadlines, and priority levels.
 - Implement drag-and-drop task reordering.
 - Add export features (PDF or text file).
-

References

- Python Tkinter Official Documentation
 - Class code and course instructions
- BuildYourOwnProject
- My original implementation code